Table des matières

Exercices corrigés 07 — Traitement distribué avec Spark 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 3
Mises en situation 3
Corrigé 5

Exercices corrigés 07 — Traitement distribué avec Spark

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur les scripts PySpark que vous venez d'écrire et sur les mesures que vous avez sous les yeux ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 07 et du Labo 07.

Les chiffres à retrouver. Plusieurs énoncés demandent une mesure faite pendant le labo, sur des résultats que son étape 8 efface. Si vous avez déjà remis le projet dans l'état du Cours 06, refaites les étapes 1 et 2 du labo, puis 3.6 et 5.2 : les scripts de Spark sont au dépôt, seuls leurs résultats manquent. Remettez ensuite le projet d'aplomb par 8.2 à 8.5.

Où écrire. Le code PySpark d'une question pratique s'écrit dans `comparaison\exercice.py`, hors du dépôt : partez de `pieges.py` pour une question qui lit le bronze, de `agregats_usine.py` pour une question qui écrit les agrégats. Il se lance dans la fenêtre 2, par `bash 01_de/spark/spark.sh comparaison/exercice.py`. Les requêtes SQL vont dans l'interface web, fenêtre 3.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les pièces. Nommez le rôle du pilote et celui d'un exécuteur, en une phrase chacun. Dites ensuite ce que les deux deviennent quand on lance Spark en `local[*]`.
2. L'unité de travail. Qu'est-ce qu'une partition, et pourquoi son nombre commande-t-il la durée d'un traitement plus sûrement que le nombre de lignes ?
3. Vingt-sept. Le labo lit 821 fichiers de 145 Ko et obtient 27 partitions de lecture. Expliquez ce nombre à partir des deux réglages qui le produisent.

© 2026 iAlexMG. Tous droits réservés.

4. Paresseux. Classez ces appels en transformation ou action : `filter`, `count`, `withColumn`, `write.parquet`, `groupBy`, `show`, `join`, `collect`.
5. Deux fois le travail. Un script fait `df.count()` puis `df.write.parquet(...)` sur le même jeu. Combien de fois les fichiers sources sont-ils lus, et que faudrait-il écrire pour n'en lire qu'une ?
6. Étroit et large. Donnez deux opérations qui ne brassent pas, et trois qui brassent. Dites en une phrase ce que « brasser » veut dire concrètement.
7. Deux cents. D'où vient le nombre 200 dans la ligne `Exchange hashpartitioning(TagName#0, _groupingexpression#20, 200)` du plan, et qu'est-ce qui fait qu'on n'obtient pourtant que trois fichiers en sortie ?
8. Les deux agrégats. Le plan des agrégats montre deux `HashAggregate` autour de l'`Exchange`. Qu'est-ce que le premier fait, et que se passerait-il s'il n'existait pas ?
9. Le codec. Le même bronze pèse 12,7 Mo d'un côté et 18,8 Mo de l'autre, pour les mêmes lignes. Expliquez l'écart, et dites lequel des deux vous garderiez pour un lac conservé cinq ans.
10. Ramener au pilote. `show(20)`, `take(20)`, `limit(20)` et `collect()` demandent tous des lignes. Lequel est dangereux sur un gros jeu, et pourquoi les trois autres ne le sont pas.

Questions pratiques

1. [labo] Les trois qualités. Écrivez le code PySpark qui compte les points du bronze par code de qualité, et dites ce qu'il doit rendre sur l'historien complet.
1. [labo] En zstd. Modifiez l'écriture des agrégats pour qu'elle emploie zstd plutôt que snappy. Donnez la ligne changée et le gain que vous attendez en octets.
2. [labo] Huit partitions. Écrivez la ligne qui ramène le brassage à huit partitions, dites combien de fichiers vous attendez en sortie, et pourquoi ce n'est pas forcément huit.
3. [labo] Sans l'exécution adaptative. Écrivez la ligne qui la désactive, puis dites ce que le dossier de sortie contiendra et ce que cela coûtera en octets et en secondes.
4. [labo] La journée la plus chargée. Écrivez le code PySpark qui donne les trois journées qui portent le plus de points, et le résultat attendu.
5. [labo] Une étiquette, un jour. Écrivez le code qui affiche les dix premiers points de `UTL.CMP2.VIBRATION` du 2025-10-28, dans l'ordre du temps, sans rien ramener d'autre au pilote.
6. [labo] Le contrôle qui tolère. Écrivez la requête DuckDB qui compare les deux jeux d'agrégats en acceptant un écart d'un millième sur la moyenne, et qui rend le nombre de lignes hors tolérance.
7. [labo] Un seul fichier. On vous demande que les agrégats de Spark sortent en un seul fichier, comme ceux du moteur d'une machine. Donnez la façon de le faire, et le prix à payer.

© 2026 iAlexMG. Tous droits réservés.

Diagnostics

1. Un collègue mesure Spark ainsi : il chronomètre les lignes `spark.read.csv(...)` et `.filter(...)`, trouve trois millisecondes, et conclut que Spark est plus rapide que DuckDB. Où est l'erreur ?
2. Un script écrit le bronze, puis relit le dossier pour compter, puis écrit un second dérivé à partir du même DataFrame de départ. Il met trois fois plus longtemps que prévu, sans rien faire d'anormal dans les journaux. Que s'est-il passé, et que changez-vous ?
3. Une ingénieure ajoute `.repartition("TagName")` avant son `groupBy` « pour aider Spark à regrouper ». Le résultat est juste et la durée double. Expliquez-lui ce qui se passe, en nommant ce que le plan montrerait.
4. Un contrôle compare les moyennes de DuckDB et de Spark avec `=` et signale six écarts. Un collègue en conclut que Spark a un défaut de calcul. Que répondez-vous, et comment écrivez-vous le contrôle correctement ?
5. Un traitement tourne pendant douze minutes, puis échoue sur `java.lang.OutOfMemoryError` dans le pilote. Le code se termine par `resultat = df.collect()` suivi d'une boucle Python qui écrit un CSV. Corrigez, sans changer ce que le script produit.
6. Un DAG du Cours 06 appelle un script PySpark par `BashOperator`, et la tâche reste bloquée. Dans les journaux d'Airflow, une seule ligne : `docker: command not found`. Où est le problème, et quelles sont les deux façons de le régler ?

Questions à développer

A. Ce cours conclut que Boréal n'a pas besoin de Spark, et fait quand même écrire quatre scripts PySpark. Défendez cette décision pédagogique, puis donnez l'argument le plus fort de celui qui la conteste.

B. Le labo mesure des rapports de 17 à 55 entre les deux moteurs, sur une seule machine. Expliquez ce que cette comparaison prouve et ce qu'elle ne prouve pas, puis dites quelles mesures il faudrait pour trancher honnêtement le jour où le volume aura été multiplié par cent.

C. Un lac de données se dégrade souvent en un dossier de dizaines de milliers de petits fichiers. Expliquez comment un moteur distribué mal réglé y contribue, ce que cela coûte à la lecture, et deux façons de l'éviter dès l'écriture.

Mises en situation

1. La direction de Boréal a lu un article sur le traitement distribué et demande une grappe Spark « pour être prêts ». On vous demande une réponse écrite d'une page. Que mettez-vous dedans, dans quel ordre, et quelle recommandation ?
2. Les Papeteries de Kamouraska exploitent quarante usines instrumentées et gardent huit ans d'historien, soit environ 8 To en Parquet. Elles vous demandent si le moteur d'une machine peut suffire. Quelles questions posez-vous avant de répondre, et quels seuils regardez-vous ?
3. Vous arrivez chez un employeur qui a déjà une plateforme Databricks et trois ingénieurs qui la connaissent. Le volume ne justifie pas le distribué. Recommandez-vous de sortir un traitement de la plateforme pour le mettre sur un moteur d'une machine ? Défendez votre position.

© 2026 iAlexMG. Tous droits réservés.

4. Une équipe voisine vous demande de reprendre son traitement PySpark, qui met quarante minutes chaque nuit. Vous n'avez accès qu'au code et à l'interface de Spark. Décrivez votre méthode, dans l'ordre, et les trois premières choses que vous regardez.
5. Un ingénieur propose d'écrire tout le pipeline de Boréal en PySpark, « pour n'avoir qu'un seul outil du bronze au capstone ». Donnez les deux options, ce que chacune coûte en exploitation, et votre recommandation pour cette entreprise-là.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Chaque réponse cite la section de `Theorie_07_DE.md` qui la fonde, ou l'étape du `Labo_07_DE.md` qui la produit. Les chiffres sont ceux d'un projet généré avec `--jusqua 2026-03 --historien complet`, comme à l'étape 1 du labo : 821 jours d'historien et 2 407 796 lignes.

Questions courtes

1. Le pilote exécute votre programme, construit le plan, découpe le travail en tâches et reçoit les résultats demandés ; il ne traite aucune donnée lui-même. Un exécuteur est un processus qui traite une partition à la fois et garde en mémoire ce qu'on lui a dit de garder. En `local[*]`, les deux vivent dans le même processus, sur tous les cœurs de la machine, douze chez l'auteur : c'est le même moteur, le même plan et le même brassage, sans le réseau.

Théorie : `Theorie_07_DE.md`, section 2.

2. Une partition est la part de données qu'une tâche traite du début à la fin, sans jamais regarder les autres. Son nombre commande la durée parce qu'il fixe le nombre de tâches, donc le parallélisme possible et la quantité de travail par cœur. Deux millions de lignes en une seule partition occupent un cœur et en laissent onze inactifs ; les mêmes lignes en dix mille partitions paient dix mille ouvertures pour rien.

Théorie : `Theorie_07_DE.md`, section 2.

3. Une partition vise 128 Mo (`spark.sql.files.maxPartitionBytes`) et l'ouverture d'un fichier compte pour 4 Mo dans ce calcul (`spark.sql.files.openCostInBytes`). Un fichier de 145 Ko coûte donc presque uniquement son ouverture, soit un peu plus de 4 Mo : Spark en met une trentaine par partition, et 821 fichiers donnent 27 partitions.

Théorie : `Theorie_07_DE.md`, section 2.

4. Transformations : `filter`, `withColumn`, `groupBy`, `join`. Actions : `count`, `write.parquet`, `show`, `collect`. Une transformation allonge le plan sans rien lire ; une action déclenche tout le plan.

Théorie : `Theorie_07_DE.md`, section 3.

5. Deux fois : chaque action réexécute le plan depuis les fichiers. Pour n'en lire qu'une, il faut demander à Spark de garder le résultat, par `df.cache()` ou `df.persist()` avant la première action. Un moteur d'une machine qui a écrit sa table ne la recalcule pas ; Spark ne garde rien qu'on ne lui ait demandé de garder.

Théorie : `Theorie_07_DE.md`, section 3.

6. Ne brassent pas : un filtre, une projection, un calcul de colonne, la lecture d'un fichier. Brassent : un groupement, une jointure, un tri global, une fonction de fenêtrage, un `repartition`. Brasser veut dire qu'une ligne change de partition : Spark écrit les partitions sur disque, redistribue les morceaux par le réseau, et relit.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_07_DE.md`, section 4.

7. Le 200 vient de `spark.sql.shuffle.partitions`, le nombre de partitions que tout brassage produit par défaut. On n'obtient que trois fichiers parce que l'exécution adaptative (`spark.sql.adaptive.enabled`, active par défaut) regarde la taille réelle des partitions après le brassage et replie les petites ensemble avant l'écriture.

Théorie : `Theorie_07_DE.md`, section 4.

8. Le premier `HashAggregate` agrège à l'intérieur de chaque partition et rend des résultats partiels, quelques milliers de lignes au lieu de 2,4 millions de points. Seuls ces résultats partiels traversent le brassage. Sans lui, toutes les lignes brutes devraient être redistribuées avant d'être groupées, et le coût du déplacement serait celui du jeu entier.

Théorie : `Theorie_07_DE.md`, section 4.

9. Le contenu est le même ; c'est le codec qui diffère. Le script du Cours 02 demande zstd, Spark écrit en snappy par défaut (`spark.sql.parquet.compression.codec`). Snappy décompresse un peu plus vite, zstd compresse nettement mieux. Pour un lac gardé cinq ans, zstd : l'espace se paie tous les mois, la décompression se paie à la lecture, et la lecture d'un lac bronze est rare.

Théorie : `Theorie_07_DE.md`, section 6.

10. `collect()` est le dangereux : il ramène toutes les lignes dans la mémoire du pilote. Les trois autres bornent ce qu'ils demandent, et Spark s'arrête dès qu'il a de quoi répondre : `show(20)` et `take(20)` rendent vingt lignes, `limit(20)` rend un jeu de vingt lignes qu'on peut encore transformer.

Théorie : `Theorie_07_DE.md`, section 7.

Questions pratiques

1. Le code, et son attendu :

```
bronze.groupBy("Quality").count().orderBy("Quality").show()
```

Attendu sur l'historien complet : 2 167 points de qualité 0, 263 de qualité 64, et 2 405 366 de qualité 192, soit 2 407 796 en tout. Ce sont les trois codes OPC du dictionnaire de l'univers.

Labo : `Labo_07_DE.md`, étape 5.

2. La ligne change d'une option :

```
agregats.write.mode("overwrite").option("compression", "zstd").parquet(SORTIE)
```

Mesuré sur les agrégats du labo : 449 473 octets en zstd contre 557 179 en snappy, pour les mêmes trois fichiers, soit environ 19 % de moins.

Théorie : `Theorie_07_DE.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

3. La ligne, avant le traitement :

```
spark.conf.set("spark.sql.shuffle.partitions", 8)
```

Huit partitions de brassage donnent au plus huit fichiers, et le labo en obtient deux : l'exécution adaptative replie les partitions trop petites avant l'écriture. Le nombre demandé est un plafond, pas une promesse.

Théorie : `Theorie_07_DE.md`, section 4.

4. La ligne qui désactive l'exécution adaptative :

```
spark.conf.set("spark.sql.adaptive.enabled", False)
```

Le dossier contient alors 200 fichiers, un par partition de brassage, pour 1 357 384 octets au lieu de 557 179, et le traitement met 34,5 s au lieu d'une dizaine. L'écart d'octets vient des en-têtes et des statistiques que chaque fichier Parquet répète : 200 fichiers minuscules coûtent plus cher que trois fichiers raisonnables, à contenu égal.

Théorie : `Theorie_07_DE.md`, sections 4 et 6.

5. Le code, et son attendu :

```
(bronze.groupBy("jour").count().orderBy(F.desc("count")).limit(3).show())
```

Attendu : le 2025-10-24 avec 4 612 points, le 2025-10-23 avec 4 352, le 2025-10-27 avec 4 297. Ces trois journées tombent pendant la dérive du compresseur d'air 2, quand la vibration franchit la bande morte à chaque scrutation : un capteur qui s'affole écrit plus de points, et cela se voit dans le volume avant de se voir dans les valeurs.

Labo : `Labo_07_DE.md`, étape 7.

6. Le code, avec un `show` et non un `collect` :

```
(bronze.filter((F.col("TagName") == "UTL.CMP2.VIBRATION") &
               (F.col("jour") == "2025-10-28"))
       .orderBy("Timestamp")
       .select("Timestamp", "Value", "Quality")
       .show(10, truncate=False))
```

La première ligne rendue porte l'horodatage du 2025-10-27 à 23:32 : c'est la valeur de départ, la dernière valeur retenue la veille, que l'historien répète en tête de chaque fichier quotidien. Le filtre sur `jour` est un filtre de partition : Spark ne lit qu'un dossier sur 821.

Labo : `Labo_07_DE.md`, étape 7.

7. La requête, et son piège :

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Les agrégats dont la moyenne s'écarte de plus d'un millième.
-- Attendu : 0 ligne, à condition d'arrondir l'écart avant de le comparer.
WITH une_machine AS (SELECT etiquette, jour, moyenne FROM 'comparaison/duckdb/agregats.parquet'),
     spark AS (SELECT TagName AS etiquette, jour, moyenne FROM 'comparaison/spark/agregats/*.parquet')
SELECT COUNT(*) AS hors_tolerance
FROM une_machine JOIN spark USING (etiquette, jour)
WHERE ROUND(ABS(une_machine.moyenne - spark.moyenne), 3) > 0.001;
```

Écrite sans l'arrondi, avec `ABS(...) > 0.001`, la même requête rend 3 et non 0 : la soustraction de deux flottants rend par exemple `-0.0010000000000000009`, qui dépasse strictement la tolérance. La tolérance doit être arrondie au même grain que les valeurs comparées, ou élargie exprès.

Labo : `Labo_07_DE.md`, étape 5.

8. Deux façons, et elles ne coûtent pas la même chose. `coalesce(1)` ramène le résultat en une seule partition sans brassage complet, mais la dernière étape n'a plus qu'une tâche, donc un seul cœur écrit. `repartition(1)` fait la même chose avec un brassage en plus. Dans les deux cas, le prix est le même : le parallélisme de l'écriture disparaît, et la mémoire d'un exécuteur doit tenir la partition entière. Sur 47 618 lignes, c'est sans conséquence ; sur un jeu qui justifierait Spark, c'est le geste qui fait tomber le traitement.

Théorie : `Theorie_07_DE.md`, sections 6 et 7.

Diagnostics

1. Il chronomètre des transformations, qui ne calculent rien : elles allongent le plan et rendent la main tout de suite. Les trois millisecondes mesurent la construction du plan, pas la lecture des fichiers. Il faut chronométrer une action, et nommer ce qu'elle déclenche : c'est ce que font les `time.perf_counter()` du labo, placés autour des écritures.

Théorie : `Theorie_07_DE.md`, section 3.

2. Chaque action repart des fichiers sources, donc le plan a été exécuté trois fois : une fois pour l'écriture, une fois pour le comptage, une fois pour le second dérivé. Il faut soit garder le résultat en mémoire par `cache()` avant la première action, soit repartir du dossier écrit, ce que font les scripts du labo en relisant le Parquet plutôt qu'en réutilisant le DataFrame de lecture.

Théorie : `Theorie_07_DE.md`, section 3.

3. Le `groupBy` provoque déjà un brassage, et il le fait après avoir agrégé dans chaque partition. Le `repartition` ajoute un brassage complet des lignes brutes avant cela : deux déplacements au lieu d'un, le premier portant tout le jeu. Le plan montrerait deux `Exchange` au lieu d'un, dont un `hashpartitioning` sous les `HashAggregate` partiels. Mesuré au labo : 17,4 s contre 8,5 s chez l'auteur.

Théorie : `Theorie_07_DE.md`, section 7.

© 2026 iAlexMG. Tous droits réservés.

4. Aucun des deux moteurs ne se trompe. Une somme de nombres à virgule flottante dépend de l'ordre des additions, et les deux moteurs ne découpent pas le travail pareil ; sur six moyennes de 47 618, le résultat exact tombe assez près d'un demi-millième pour que l'arrondi bascule. Le contrôle correct compare avec une tolérance arrondie (`ROUND(ABS(a - b), 3) > 0.001`), ou porte sur les comptes et les extrêmes, qui sont exacts.

Labo : `Labo_07_DE.md`, étape 5.

5. Le `collect()` ramène tout le jeu dans le pilote, et c'est lui qui tombe. La correction consiste à laisser Spark écrire : `df.write.mode("overwrite").option("header", True).csv(chemin)`, qui produit un dossier de fichiers CSV écrits en parallèle. Si le livrable doit être un fichier unique, `df.coalesce(1).write...`, en sachant qu'un seul cœur écrit et qu'une partition doit tenir en mémoire. La boucle Python disparaît dans les deux cas.

Théorie : `Theorie_07_DE.md`, section 7.

6. Le `BashOperator` s'exécute à l'intérieur d'un conteneur Airflow, qui n'a ni Docker ni accès au moteur de l'hôte. Deux façons de régler : donner au conteneur d'Airflow l'accès au moteur Docker de l'hôte, par le montage de sa socket et un opérateur prévu pour cela, ce qui revient à confier les clés de la machine à l'orchestrateur ; ou ne pas lancer Spark depuis Airflow, et faire appeler par la tâche une commande qui soumet le travail à un service Spark existant. Pour Boréal, la question ne se pose pas : le pipeline du Cours 05 fait le travail, et Spark n'est là que pour la comparaison.

Labo : `Labo_07_DE.md`, étape 3.

Questions à développer

A. La décision se défend sur trois points. Le premier est l'employabilité : PySpark est demandé dans les offres, et ne pas savoir l'écrire ferme des portes que le volume de Boréal, lui, n'ouvre ni ne ferme. Le deuxième est le vocabulaire : partition, brassage, plan d'exécution et déséquilibre servent à lire n'importe quel moteur moderne, y compris ceux d'une seule machine. Le troisième est le jugement : on ne peut pas dire « nous n'en avons pas besoin » de façon crédible sans avoir mesuré les deux, et c'est cette phrase, chiffres à l'appui, qui vaut en entrevue. L'argument contraire le plus fort est celui du temps : ces quatre scripts et les deux séances du labo auraient pu aller à la qualité des données ou aux contrats, qui manquent plus souvent dans une équipe qu'un moteur distribué. Un cours est un budget, et ce budget-là a été mis sur un outil qu'on recommande de ne pas employer.

Théorie : `Theorie_07_DE.md`, section 8.

B. La comparaison prouve une chose solide : sur ce volume et sur cette machine, le moteur d'une machine fait le même travail avec les mêmes résultats, deux ordres de grandeur plus vite, et le coût de démarrage du distribué n'y est pour presque rien puisqu'il est mesuré à part. Elle ne prouve rien sur le comportement à cent fois le volume, où Spark étale le travail sur plusieurs machines pendant que le moteur d'une machine bute sur sa mémoire et sur son disque. Pour trancher honnêtement le jour venu, il faudrait mesurer sur le volume réel, pas sur un multiple estimé, et comparer sur trois axes : la durée à machine égale, la durée à coût égal, et la tenue du traitement quand une machine tombe en route. La dernière est souvent celle qui décide, et elle ne se mesure pas en secondes.

Théorie : `Theorie_07_DE.md`, section 8.

C. Un moteur distribué écrit un fichier par partition, et le nombre de partitions vient d'un réglage qui ignore la taille du résultat. Un brassage à 200 partitions sur un résultat de 50 000 lignes donne 200 fichiers de quelques kilooctets, et un traitement quotidien en crée 200 de plus chaque jour. À la lecture, chaque fichier

© 2026 iAlexMG. Tous droits réservés.

coûte une ouverture, la lecture de son pied de page et de ses statistiques ; sur un stockage objet, c'est une requête réseau par fichier, et le temps de réponse domine tout. Le labo le mesure à petite échelle : 200 fichiers pèsent 1 357 384 octets là où trois en pèsent 557 179, pour le même contenu. Deux façons de l'éviter dès l'écriture : laisser l'exécution adaptative replier les partitions, et régler le nombre de partitions de brassage sur la taille attendue du résultat plutôt que de garder 200 ; puis, pour ce qui s'accumule dans le temps, prévoir un compactage régulier, que les formats de table du Cours 10 savent faire.

Théorie : `Theorie_07_DE.md`, sections 4 et 6.

Mises en situation

1. La page tient en quatre parties. D'abord ce qui a été mesuré, et sur quoi : 2 407 796 points, deux moteurs, les mêmes résultats, 2,7 s contre 46 à 77 s. Ensuite ce que la mesure ne dit pas : elle vaut pour ce volume et cette machine, et le jour où une deuxième usine s'ajoute, on remesure. Puis ce qu'une grappe coûterait vraiment, en exploitation plutôt qu'en licence : des services à surveiller, des versions à suivre, une compétence à garder dans l'équipe. Enfin la recommandation : garder le moteur d'une machine, garder les scripts PySpark au dépôt pour que le jour venu le travail soit déjà écrit, et refaire la mesure quand le volume aura été multiplié par dix. Une réponse qui dit non sans chiffres se fait renverser au conseil suivant.

Théorie : `Theorie_07_DE.md`, section 8.

2. Les questions d'abord. Combien de données sont réellement lues par les traitements quotidiens, plutôt que gardées : huit téraoctets d'archives dont on ne relit que le dernier mois ne sont pas huit téraoctets de calcul. Quel est le temps imparti, et qui l'a fixé. Combien de traitements tournent en même temps, et pour combien d'équipes. Où vit la donnée : sur un disque local, ou sur un stockage objet où le calcul doit se déplacer. Quelle compétence existe déjà. Les seuils que je regarderais : le volume lu par exécution face à la mémoire d'une machine, la durée mesurée face à la fenêtre disponible, et la tolérance à la panne d'une machine en cours de traitement. Avec quarante usines, le partitionnement par usine et par jour changerait probablement plus de choses que le moteur.

Théorie : `Theorie_07_DE.md`, section 8.

3. Non, et pour une raison qui n'est pas technique. Le coût d'un traitement ne se limite pas à sa durée : il comprend la personne qui le reprendra dans deux ans, l'environnement où il tourne, la surveillance qui l'entoure. Sortir un traitement d'une plateforme que trois personnes maîtrisent pour l'isoler sur un outil que personne n'exploite crée une exception coûteuse, qui se paiera à la première panne un vendredi soir. La bonne recommandation est de rester sur la plateforme, et d'y appliquer ce que le cours enseigne : régler les partitions de brassage, éviter les redistributions inutiles, surveiller le nombre de fichiers écrits. Le jugement change si la facture de calcul devient un sujet, et là encore, cela se mesure.

Théorie : `Theorie_07_DE.md`, section 8.

4. La méthode, dans l'ordre. Lire le code en cherchant les actions : leur nombre dit combien de fois le plan est exécuté, et c'est la cause la plus commune d'une durée triplée. Puis afficher le plan du traitement le plus lourd et compter les `Exchange` : chacun est un déplacement complet, et un de trop se voit tout de suite. Puis regarder, dans l'interface de Spark, la durée des tâches d'une même étape : quand la plus longue fait dix fois la médiane, le problème est un déséquilibre de partition, pas le moteur. Les trois premières choses que je regarde sont donc le nombre d'actions, le nombre d'échanges, et l'écart entre la tâche la plus longue et la médiane de son étape.

Théorie : `Theorie_07_DE.md`, sections 3, 4 et 7.

© 2026 iAlexMG. Tous droits réservés.

5. Option un, tout en PySpark : un seul outil à connaître, un seul style de code, et la porte ouverte si le volume explose. En exploitation, cela veut dire une image de 2,22 Go dans chaque environnement, une machine virtuelle Java à démarrer pour chaque traitement, une vingtaine de secondes de démarrage pour un travail qui prend deux secondes, et un pipeline dont le kit livré à Analytics Engineering ne se rejoue plus sans Spark. Option deux, garder DuckDB pour le pipeline et Spark pour ce qu'il est, un cours et une réserve : deux outils à connaître, mais un seul à exploiter, et un kit qui se rejoue avec Python et la bibliothèque standard. Pour Boréal, la seconde. Le critère décisif n'est pas la vitesse, c'est que le chemin du kit reste rejouable par la formation suivante sans installer une plateforme.

Théorie : `Theorie_07_DE.md`, section 8.

© 2026 iAlexMG. Tous droits réservés.